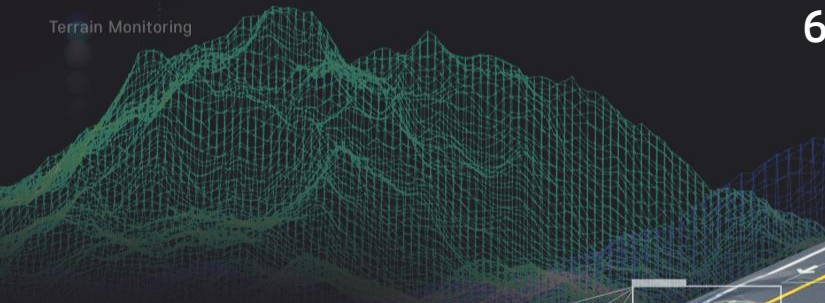


Image Recognition



Terrain Monitoring



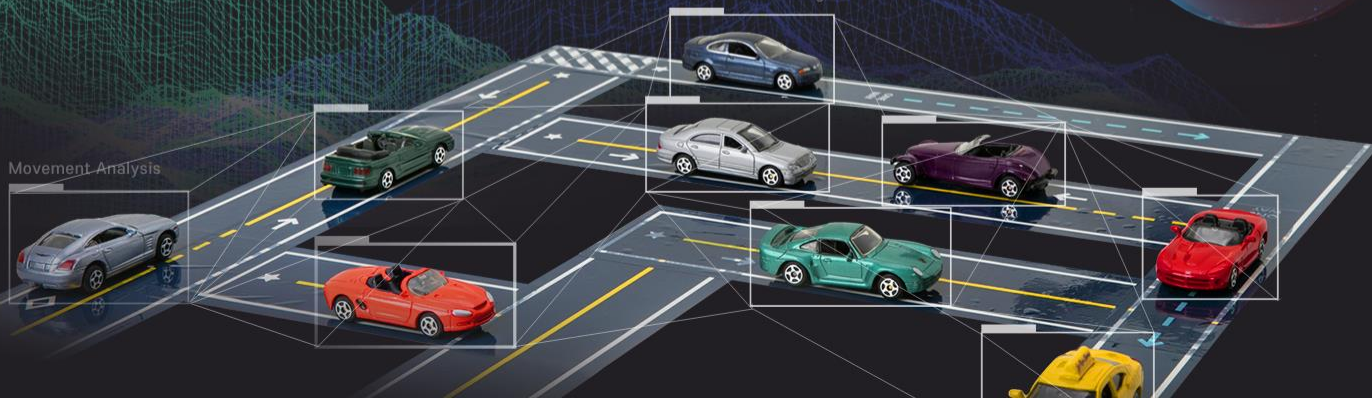
강원혁신플랫폼

컴퓨터 비전

6개의 클래스 분류

Autonomous Driving

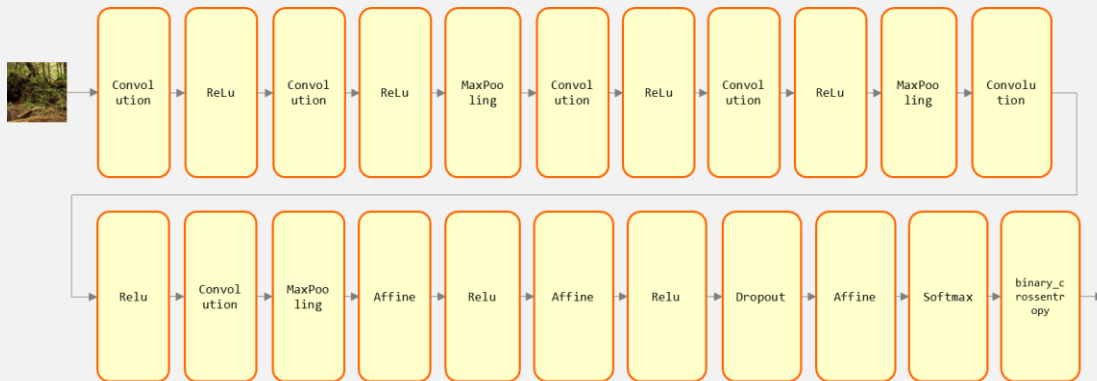
Movement Analysis



학습된 인공지능망 모델을 읽어와서 테스트 데이터 셋으로 모델을 평가 및 추론을 수행해보자.

- 검증 데이터 셋의 25장 이미지로 분류 예측을 수행해 그 결과 이미지를 출력한다.
- 이를 위해 학습된 모델 “Intel_image_classification_model.h5” 파일을 메모리에 읽어온다.
- 즉, 앞에서 생성한 9층 인공지능망 네트워크 구조에 학습된 매개변수로 초기화한다.

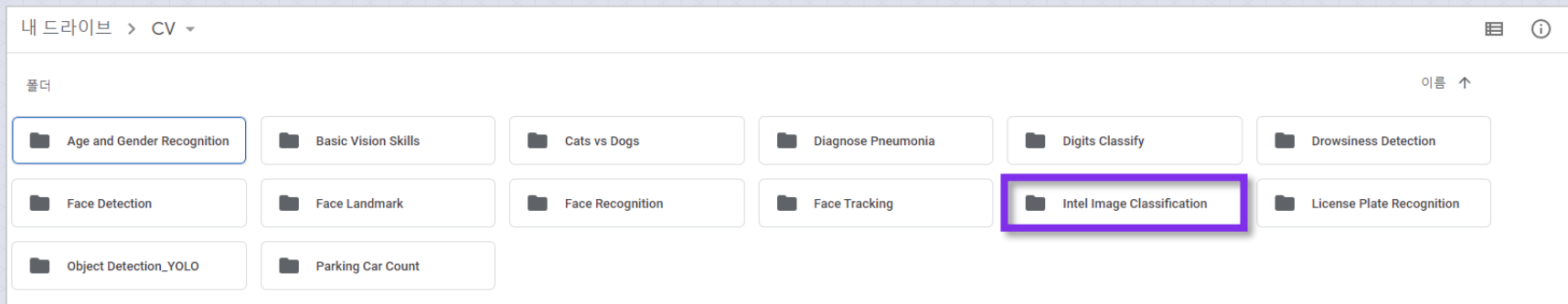
9층 인공지능망 모델의 계층구조로 이루어진 네트워크 구조



인공지능망 네트워크 구성

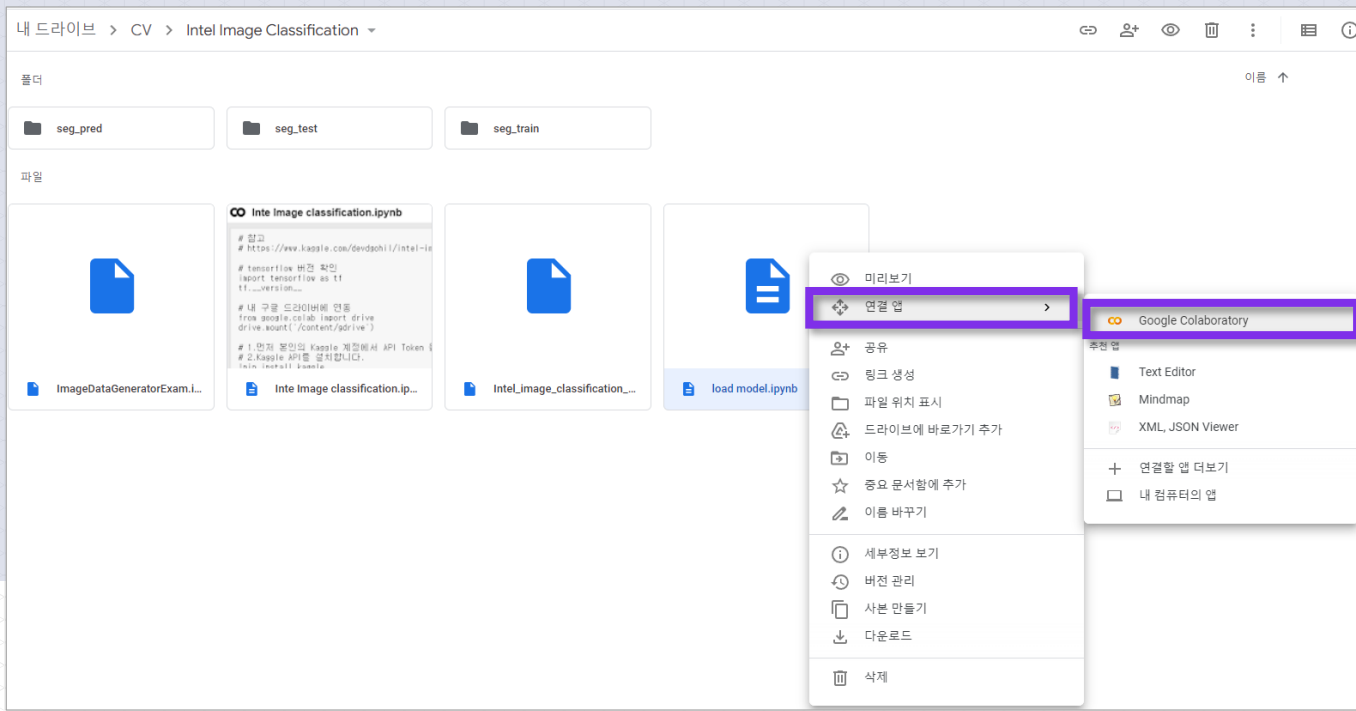
① 실습 파일 복사하기

- ➡ 학습된 모델을 이용해 추론을 수행해보자.
- ➡ 구글 드라이브 > CV > Intel Image Classification 폴더를 클릭한다.

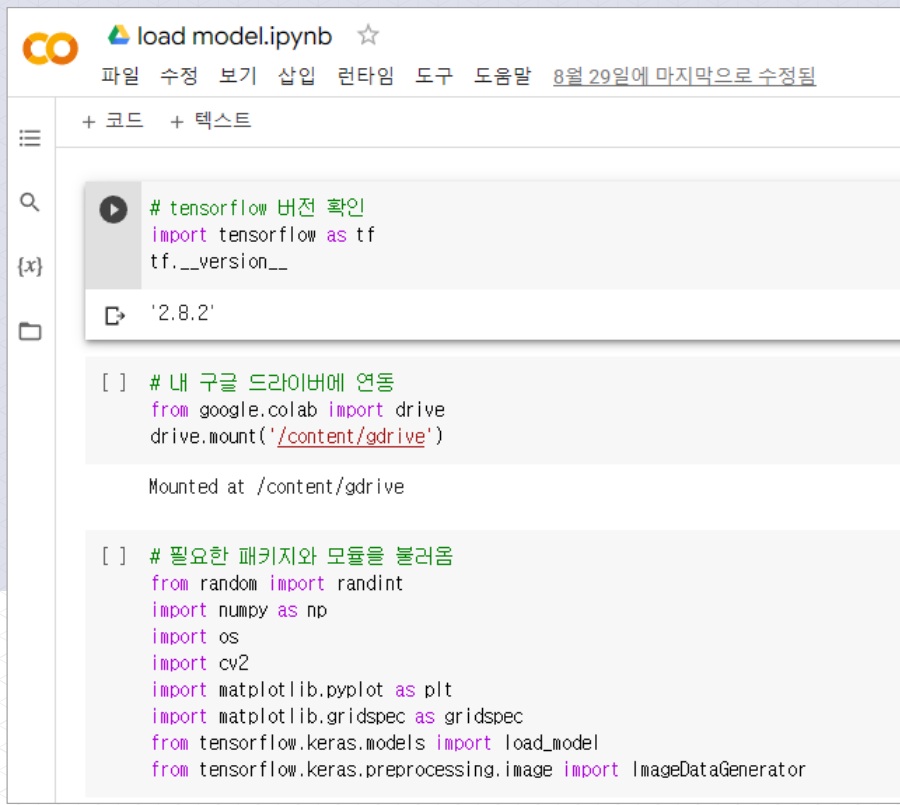


① 실습 파일 복사하기

- 아래의 그림의 Intel Image Classification 폴더에서 **Load model.ipynb**를 선택 후 오른쪽 마우스를 클릭하여 연결 앱 > **Google Colaboratory**을 클릭한다.



아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



```
load model.ipynb ☆
파일 수정 보기 삽입 런타임 도구 도움말 8월 29일에 마지막으로 수정됨

+ 코드 + 텍스트

# tensorflow 버전 확인
import tensorflow as tf
tf.__version__

'2.8.2'

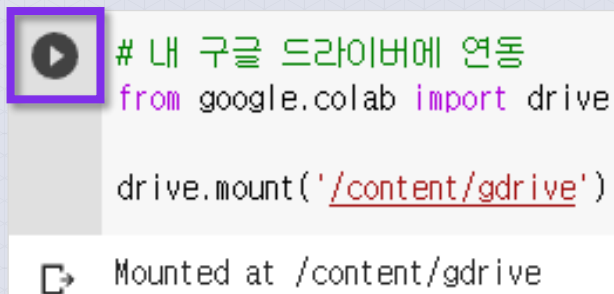
[ ] # 내 구글 드라이브에 연동
from google.colab import drive
drive.mount('/content/gdrive')

Mounted at /content/gdrive

[ ] # 필요한 패키지와 모듈을 불러옴
from random import randint
import numpy as np
import os
import cv2
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

② 구글 드라이브에 연동하기

- ☞ 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.



② 구글 드라이브에 연동하기

- ① [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택한다.
- ② gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요 **Google Drive에 연결**

Google 계정으로 로그인

계정 선택

Google Drive for desktop(으)로 이동

장원중
wjjang2040@gmail.com

② 다른 계정 사용

계속 진행하기 위해 Google에서 내 이름, 이메일 주소, 언어 환경 설정, 프로필 사진을 Google Drive for desktop과(와) 공유합니다. 앱을 사용하기 전에 Google Drive for desktop의 개인정보처리방침 및 서비스 약관을 검토하세요.

한국어 ▼ 도움말 개인정보처리방침 약관

구글 드라이브 접근 허용

Google 계정으로 로그인

Google Drive for desktop에서 내 Google 계정에 액세스하려고 합니다

wjjang2040@gmail.com

어떻게 하면 Google Drive for desktop에서 내 작업을 할 수 있습니까?

- Google Drive 파일 보기, 수정, 생성, 삭제
- Google 포토의 사진, 동영상, 앨범을 봅니다.
- 모바일 클라이언트로 구성 및 실행 정보 가져오기
- 프로필 및 연락처와 같은 Google 사용자 정보 조회
- Google Drive 파일 작업 기록 조회
- Google Drive 문서 보기, 수정, 생성, 삭제

Google Drive for desktop 앱을 실행할 수 있는지 확인

명확한 정보가 이 사이트 또는 앱과 공유될 수 있습니다. 언제든지 Google 계정에서 액세스 권한을 확인하고 삭제할 수 있습니다.

Google이 데이터를 안전하게 공유하는 방법을 알아보세요.

Google Drive for desktop이 개인정보처리방침 및 서비스 약관을 확인하세요.

취소 **허용**

한국어 ▼ 도움말 개인정보처리방침 약관

- 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.



필요한 패키지와 모듈을 불러옴

```
from random import randint
```

```
import numpy as np
```

```
import os
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib.gridspec as gridspec
```

```
from tensorflow.keras.models import load_model
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```


- 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.
- 학습된 모델**은 Intel Image Classification 폴더 아래의 **"Intel_image_classification_model.h5"** 파일에 저장되어 있다.

```

1 # 학습된 모델을 읽어온다.
2
3 new_model = load_model('gdrive/My Drive/CV/Intel Image Classification/Intel_image_classification_model.h5')
4 new_model.summary()
    
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1 (Conv2D)	(None, 148, 148, 64)	1792
conv2 (Conv2D)	(None, 146, 146, 64)	36928
maxpool1 (MaxPooling2D)	(None, 48, 48, 64)	0
conv3 (Conv2D)	(None, 46, 46, 128)	73856
conv4 (Conv2D)	(None, 44, 44, 128)	147584
maxpool2 (MaxPooling2D)	(None, 14, 14, 128)	0
conv5 (Conv2D)	(None, 12, 12, 256)	295168
conv6 (Conv2D)	(None, 10, 10, 256)	590080
maxpool3 (MaxPooling2D)	(None, 3, 3, 256)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 1024)	2360320
dense_1 (Dense)	(None, 256)	262400
dropout (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 6)	1542

Total params: 3,769,670
 Trainable params: 3,769,670
 Non-trainable params: 0

다음은 ImageDataGenerator 클래스로 `test_datagen` 인스턴스를 만든다.

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨



```
1 # ImageDataGenerator 클래스를 통해 test_datagen 인스턴스를 만듭니다.  
2 test_datagen = ImageDataGenerator(rescale=1./255) # 픽셀 값을 0~1 범위로 변환
```

- tensorflow.keras의 ImageDataGenerator 클래스는 데이터 증강(Data Augmentation)을 지원함
- 위 코드에서는 이미지 각 픽셀 값을 0~1 범위로 변환함

ImageDataGenerator 클래스를 통해 테스트 데이터 셋을 이용해 한번에 `batch_size` 만큼 이미지를 만들어내는 `test_generator` 객체를 생성한다.

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨



```
1 # ImageDataGenerator를 통해 validation dataset 에서 한번에 batch_size 만큼
2 test_generator = test_datagen.flow_from_directory(
3     'gdrive/My Drive/CV/Intel Image Classification/seg_test/seg_test',
4     batch_size=100,
5     class_mode='sparse',
6     target_size=(150, 150))
```

Found 3000 images belonging to 6 classes.

- `flow_from_directory()` 함수는 `directory` 경로로부터 데이터프레임을 가져오고, 증강(Augmented) 또는 정규화(Normalized)된 데이터 배치를 생성함
- 위 코드에서 테스트 데이터 셋에서 `batch_size=100` 만큼 이미지로 `train_generator` 객체가 만들어진 것을 알 수 있음
- 실행 결과 훈련 데이터 셋으로 6개의 클래스로 구성된 3,000장의 이미지가 있음을 알 수 있음

다음은 레이블 명(label name)을 찾아오는 findKey() 함수를 정의한다.

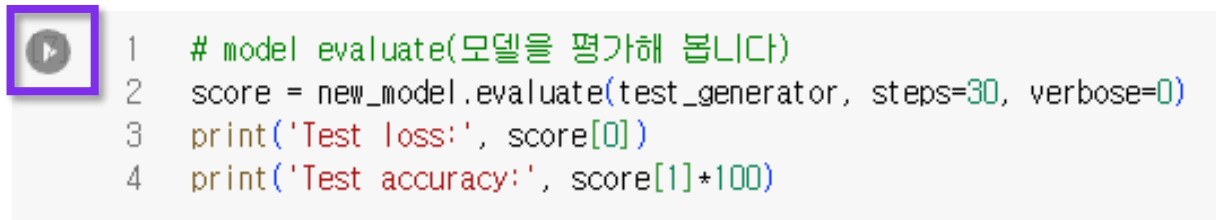
```
▶ # label name 을 찾아 오는 함수를 정의한다.  
def findKey(indices, search_value):  
    for key, value in indices.items():  
        if(value == search_value):  
            return key  
    return -1
```

- 위 코드에서 findKey() 함수는 indices 객체(딕셔너리)의 value와 search_value의 값이 동일한 경우, indices 객체의 key 값을 반환함

[예] indices = {'buildings':0, 'forest':1, 'glacier':2, 'mountain':3, 'sea':4, 'street':5},
search_value = 0인 경우, findKey() 함수의 반환값은 'building'이 됨

학습된 인공지능망 모델에 테스트 데이터 셋으로 평가 해보자.

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨



```
1 # model evaluate(모델을 평가해 봅니다)
2 score = new_model.evaluate(test_generator, steps=30, verbose=0)
3 print('Test loss:', score[0])
4 print('Test accuracy:', score[1]*100)
```

Test loss: 0.4883247911930084
Test accuracy: 87.56666779518127

- 실행 결과 테스트 데이터 셋에 대한 정확도가 87.566% 나온 것을 볼 수 있음

다음은 디렉토리(directory)에서 이미지 파일들을 읽어오는 함수 `get_images()`를 정의한다.

🔄 `get_images()` 함수는 지정한 디렉토리에서 25장의 이미지를 읽어옴

```
# 디렉토리에서 이미지 파일들을 읽어오는 함수 정의
def get_images(directory):
    Images = []
    pred_img_cnt = 25 # 예측할 이미지를 몇개 불러 올 것인지 설정
    i = 0
    for image_file in os.listdir(directory): # 디렉토리로부터 파일을 하나하나 불러 온다
        i += 1
        if i > pred_img_cnt: break
        else:
            image = cv2.imread(directory+r'/' + image_file) # 불러온 이미지를 openCV 로 읽어 들인다
            image = cv2.resize(image, (150,150)) # 이미지 크기 재조정
            Images.append(image)

    return Images
```

예측에 사용할 이미지를 읽어온다.

```
1 # 예측 할 이미지를 불러온다
2 pred_images = get_images('gdrive/My Drive/CV/Intel Image Classification/seg_pred/seg_pred')
3 print(pred_images[0])
4 pred_images = np.array(pred_images) * 1./255
5 print(pred_images[0])
6 pred_images.shape      # (25, 150, 150, 3) : (이미지수, 세로(H), 가로(W), 채널(C))
```

- 이미지 형상을 보면 이미지 수=25장, 세로 크기=150, 가로 크기=150, 채널=3 (컬러 영상)인 것을 알 수 있음
- 아래의 그림은 첫 번째 이미지의 정규화 이전과 이후의 값을 출력한 것임

```
[[[ 84  89  98]
   [ 79  84  93]
   [ 78  80  91]
```

```
...
[220 223 221]
[215 218 216]
[224 227 225]]
```

정규화 이전(0~255)

```
[[[0.32941176 0.34901961 0.38431373]
   [0.30980392 0.32941176 0.36470588]
   [0.30588235 0.31372549 0.35686275]
```

```
...
[0.8627451  0.8745098  0.86666667]
[0.84313725 0.85490196 0.84705882]
[0.87843137 0.89019608 0.88235294]]
```

정규화 이후(0~1)

학습된 모델에 예측할 이미지로 분류 예측을 수행해 보자.

- 여기서는 25장 이미지를 분류 예측하고 그 결과를 prediction 객체에 저장함

```
# 예측할 이미지를 우리가 만든 모델을 통해 분류 예측해 봅니다.  
prediction = model.predict(pred_images, verbose=1)
```

```
1/1 [=====] - 1s 794ms/step
```

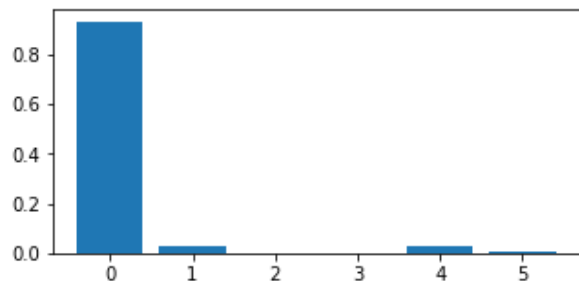
- 다음은 prediction 객체의 형상을 확인한 것이며, 형상은 (25, 6)인 것을 알 수 있음

```
1 # 예측한 결과 객체 prediction shape 확인  
2 prediction.shape
```

```
(25, 6)
```

학습된 모델에 예측할 이미지로 분류 예측을 수행해 보자.

- 다음은 prediction 객체의 예측결과를 확인한 것임
- 첫 번째 예측결과에서, 출력층의 활성화함수로 소프트맥스를 사용함으로 확률값이 출력된 것을 알 수 있음



➡ 출력층 활성화함수 : 소프트맥스

빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5

- 첫 번째 이미지는 첫 번째 인덱스의 값이 가장 큰 것을 알 수 있음(즉, 빌딩으로 예측한 것임)

```

1 # 예측결과 prediction 객체의 첫 번째 이미지 결과값
2 prediction[0]
3 #array([7.0173562e-01, 2.8287795e-01, 6.5468735e-04, 1.8440540e-05, 9.9101482e-05, 1.4614122e-02], dtype=float32)

array([7.0173562e-01, 2.8287795e-01, 6.5468735e-04, 1.8440540e-05,
       9.9101482e-05, 1.4614122e-02], dtype=float32)
    
```

test_generator 객체가 만들어 내는 **이미지**의 **클래스 인덱스**를 확인해 보자.

```
# test_generator 가 만들어 내는 이미지의 클래스 인덱스가 무엇인지 확인해 본다.  
print(test_generator.class_indices)
```

```
{'buildings': 0, 'forest': 1, 'glacier': 2, 'mountain': 3, 'sea': 4, 'street': 5}
```

🔄 실행 결과는 0:빌딩, 1:숲, 2:빙하, 3:산, 4:바다, 5:거리임을 알 수 있음

빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5

🔄 위 코드에서 **class_indices** 객체 변수의 자료구조는 **딕셔너리**인 것을 알 수 있음

다음은 학습된 모델이 예측한 25장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화해 보자.

예측결과 값은 확률값이고, 확률값이 가장 큰 인덱스 위치가 모델이 예측한 결과임

▶ # 예측한 결과는 아래와 같습니다. 이미지의 위에는 class label을 이미지 아래에는 확률을 나타냅니다.

```
fig = plt.figure(figsize=(30, 30))
outer = gridspec.GridSpec(5, 5, wspace=0.2, hspace=0.2)

for i in range(25):
    inner = gridspec.GridSpecFromSubplotSpec(2, 1, subplot_spec=outer[i], wspace=0.1, hspace=0.1)
    rnd_number = randint(0, len(pred_images)-1)
    pred_image = np.array([pred_images[rnd_number]])

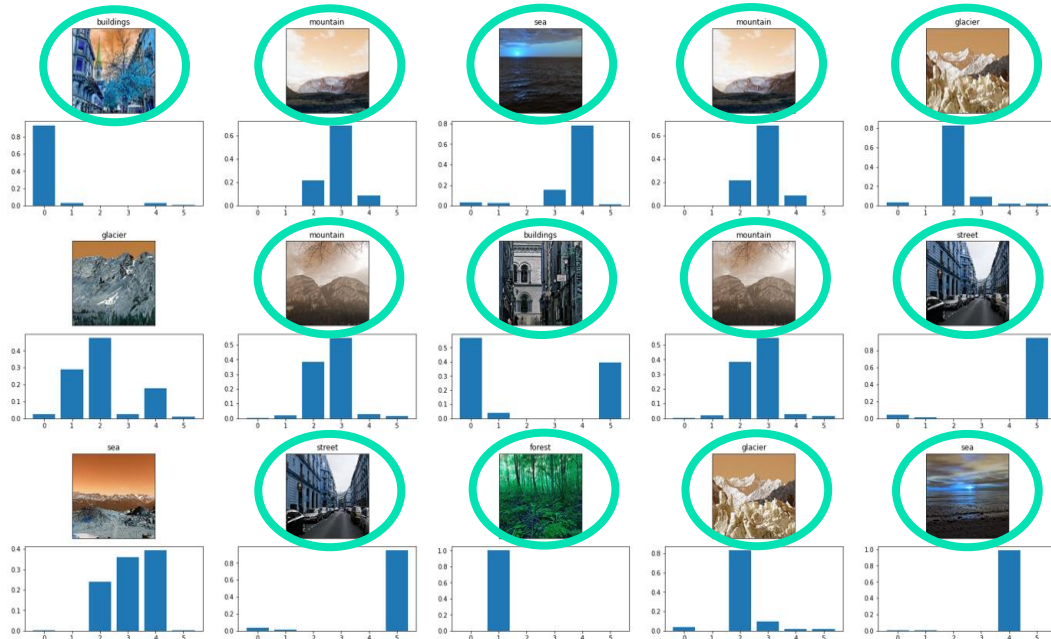
    pred_class = findKey(train_generator.class_indices, np.argmax(prediction[rnd_number])) # 이미지의 label name 을 찾아줍니다.
    pred_prob = prediction[rnd_number].reshape(6) # 예측된 결과(확률값)를 1차원 배열(6칸)로 reshape 합니다.
    for j in range(2):
        if (j%2) == 0:
            # 처음에는 이미지와 title(label name) 을 보여줍니다.
            ax = plt.Subplot(fig, inner[j])
            ax.imshow(pred_image[0])
            ax.set_title(pred_class)
            ax.set_xticks([])
            ax.set_yticks([])
            fig.add_subplot(ax)
        else:
            # 그다음은 확률값을 그래프 형태로 보여줍니다.
            ax = plt.Subplot(fig, inner[j])
            ax.bar([0,1,2,3,4,5], pred_prob)
            fig.add_subplot(ax)

fig.show()
```

다음은 학습된 모델이 예측한 25장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화해 보자.

예측결과

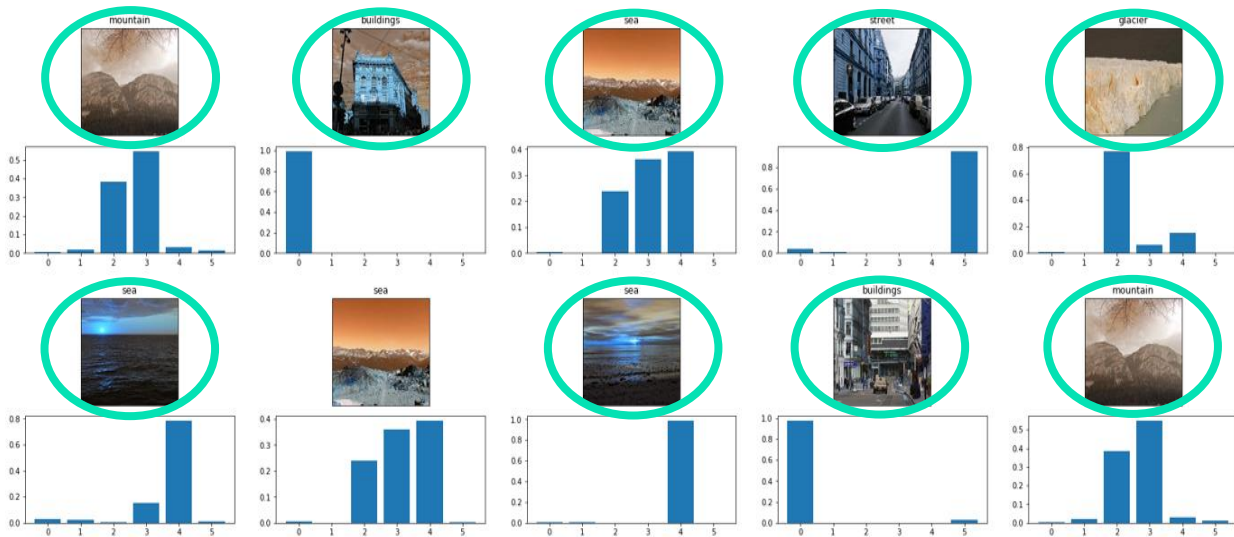
빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5



다음은 학습된 모델이 예측한 25장 이미지의 예측결과(predictions 객체에 저장된 결과)를 시각화해 보자.

예측결과

빌딩(Buildings)	숲(Forest)	빙하(Glacier)	산(Mountain)	바다(Sea)	거리(Street)
0	1	2	3	4	5



정답 : 22 / 오답 : 3